

Lab 5

Instructions:

In this lab you should use reinforcement learning to solve the maze (use the code to generate a maze from earlier Labs or the RL modules). The maze layout is stored in a list of lists where values of 0 mark hallways, and a 1 indicates a wall. The maze should be generated not smaller than 8x8, which, including walls, will result in a maze of size 17x17.

The start is consequently at location 1.1 and the goal is at 15.15.

We discussed Q-learning as one of the most important methods to solve this problem. Since the maze has a very simple reward function: Solving it is good (reward=1.0), everything else is bad or incomplete (reward=0.0), you do not have to use the complex reward function, which includes future prediction, decay, and different rewards, but instead a simple approach suffices. Note that using a decay on the rewards can accelerate finding the solution and help convergence. We recommend implementing it.

You are welcome to use the more complex approach though.

The recommended approach is to create a policy which stores probabilities, these probabilities predict the optimal path, not predicted rewards.

Deliverables:

- Ipython (or Jupyter) notebook with your ANNOTATED code. Annotated means: sporadic comments inline with the code that contextualize what the code is doing. And adding comment cells to separate logical contexts helps you understand the code.
- visualize how your code improves the time it takes for the agent to find the goal (a plot of time over epochs), additional visualizations are welcome (a print statement doesn't count as a visualization)
- A video (link or upload) where you describe your solution approach. Please keep it short (aim for 5 minutes, but for sure under 10). In the video, show how the algorithm improves the performance over time. Since the algorithm will take several minutes, run it before and show the result.

Evaluation:

- We will watch the videos and determine if you understood the concepts and what you implemented
- You are allowed to use all code developed in class or during the lab
- External libraries such as numpy, matplotlib etc. are fine, or if you really use a neural network even those libraries are acceptable (pytorch, keras). Not allowed are libraries that provide the code for reinforcement learning.